

# Position Encoding in the Euclidean Plane Using the Anoto<sup>®</sup> Codec and py-microdots

Christoph Heindl

Visual Computing, PROFACTOR GmbH, Austria  
cheind@profactor.at

**Abstract.** Determining the precise position of devices over large spaces is a challenging problem. In this work, we provide an in-depth review of the Anoto<sup>®</sup> micro-dot codec to solve this localization problem in the Euclidean plane. Starting from the preferred embodiment of the code used in many digital pen applications, we proceed to generalize the underlying principles to enable codes tailored to application-specific requirements. To facilitate working with these codes, we introduce a modern Python library named `py-microdots`<sup>1</sup> to encode, decode and customize Anoto<sup>®</sup> based patterns.

**Keywords:** positional encoding, planar localization, structured patterns

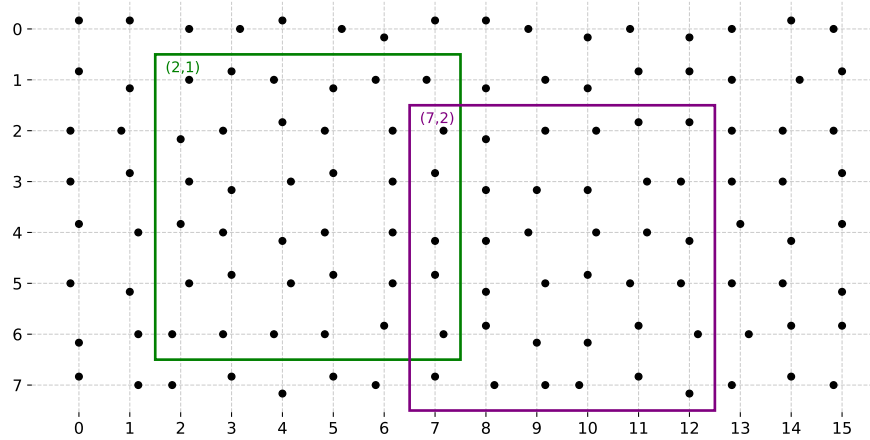


Fig. 1: **Overview.** The Anoto<sup>®</sup> structured pattern encodes a unique 2D position for every possible 6x6 sub-array of dots (green and purple rectangles with decoded locations). Theoretically, at a grid resolution of 0.3 mm, this coding remains unique over the area of Europe and Asia. For clarity, the dots are significantly scaled up and nominal grid lines are shown.

<sup>1</sup> <https://github.com/cheind/py-microdots>

## 1 Introduction

This work considers a method for localizing devices in the Euclidean plane by observing parts of a structured pattern. In its preferred embodiment, the pattern consists of microdots arranged in a grid, with each dot deliberately offset from its nominal grid position to encode 2 bits of information. Decoding the absolute device location merely requires observing any subset of  $6 \times 6$  dots as depicted in Figure 1. Assuming a grid point spacing of about 0.3 millimeters, one could theoretically code the area of Europe and Asia unambiguously. This code was developed by Anoto<sup>®</sup> and patented in the early 2000s. Because the pattern is easy to produce, leaves only a small visual footprint and requires little computer resources to decode, the technology has quickly found its way into many digital applications such in digital paper and interactive books and games.

Since its introduction, the Anoto<sup>®</sup> coding has only been studied in a handful of scientific papers, despite the fact that it seems exceptionally useful for many tasks such as indoor positioning and camera calibrations. The primary reason for this is, we suspect, that genuine information can only be found in published patent information. As existing patents are about to expire and only a few scientific papers on this versatile localization approach have been published recently, we believe it is worthwhile to revisit the Anoto<sup>®</sup> approach.

This paper is organized as follows: After discussing related work in Section 2, we highlight the mathematical concepts underpinning the central codec ideas in Section 3. Section 4 then explains the coding principles based on the preferred coding embodiment and subsequently generalizes these principles to arbitrary embodiments. In Section 5 we introduce our Python library `py-microdots` to process Anoto<sup>®</sup> based codings. Finally, we present our main findings in Section 6 and discuss further research directions in Section 7.

## 2 Related Work

This work is mostly based on genuine details given in the Anoto<sup>®</sup> patents [7–10]. In addition, the works of Aboufadel et al. [1], Hostettler et al. [4], Zhang et al. [11] and the dissertation of Özgür [12] provided helpful insights of the Anoto<sup>®</sup> codec.

We have found two open source software projects dealing with the Anoto<sup>®</sup> codec. The first one is `freedot`[3] by Flaherty and consists of a collection of C++ and Python scripts for reverse engineering existing Anoto<sup>®</sup> devices. The second library is called `libdots` [4] written in C/C++. It provides routines for encoding and decoding the preferred embodiment of the code. In contrast to our work, their work also contains an image processing pipeline tailored to a specific sensor setup. Neither of the aforementioned software packages appear to expose a generic API for handling custom embodiments of the codec.

Our contributions include (i) an in-depth explanation of the Anoto<sup>®</sup> code with many details not found in other works such as section coordinates, (ii) a generalization of the preferred embodiment to arbitrary codings and (iii) a modern

Python library to handle preferred and custom embodiments. We intentionally leave image processing for dot pattern recognition to future work, in which we intend to explore a general approach that should work well with different code embodiments and across sensor devices.

### 3 Preliminaries

This section briefly introduces mathematical concepts related to the Anoto<sup>®</sup> codec principle. The interested reader is referred to classical textbooks on Number Theory [5, 6] for more information. At this point, the following concepts may seem unrelated, but they will be tied together in Section 4 to form the Anoto<sup>®</sup> code.

#### 3.1 De Bruijn Sequences

A cyclic de Bruijn sequence of order  $n$  over an alphabet of size  $k$  is a  $k$ -ary sequence of length  $k^n$  with the following properties: (i) all possible  $k$ -ary strings of length  $n$  are contained and (ii) each string of length  $n$  appears exactly once. We denote the set of possible de Bruijn sequences for a particular configuration  $B(k, n)$ . Related to this concept is the set of cut-down or quasi de Bruijn sequences, denoted  $QB(k, n, m)$ , which contains all universal cycles of length  $1 \leq m \leq k^n$  such that any  $k$ -ary string of length  $n$  appears at most once. Examples for each sequence type are given below

$$\begin{aligned} x \in B(2, 4) &: (0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0) \\ x \in QB(2, 4, 12) &: (0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0). \end{aligned}$$

#### 3.2 Prime Factor Integer Bases

Given an integer  $n$  and its factorization into prime factors  $(p_1, \dots, p_m)$ , one may express any integer  $e \in [0, n)$  using  $m$  coefficients  $(c_1, \dots, c_m)$  with  $c_i \in [0, p_i)$  by

$$e = \sum_{i=1}^m b_i c_i \quad (1)$$

with bases  $b_i = \prod_{j=1}^{i-1} p_j$  and  $b_1 = 1$ . Using matrix notation, we may write  $e = \mathbf{b}^T \mathbf{c}$ , where  $\mathbf{b} = [b_1 \cdots b_m]$  is the basis column vector and  $\mathbf{c} = [c_1 \cdots c_m]$  is a column vector composed of coefficients. It is important to note that any permutation of the sequence  $(p_1, \dots, p_m)$  gives rise to a different sequence of bases and will hence change the coefficients. Table 1 gives an example.

Projecting the integer  $e$  onto the set of prime factor bases can be accomplished via modular arithmetic and integer division as follows. Let  $q = e$  and take the largest basis  $b_m$ . Then the quotient of integer division  $q/b_m$  equals  $c_m$ . Next, set  $q = q \bmod b_m$  and repeat with the next largest basis  $b_{m-1}$  until all bases have been processed.

|    | [2,2,3] | [2,3,2] | [3,2,2] |
|----|---------|---------|---------|
| 0  | 000     | 000     | 000     |
| 1  | 100     | 100     | 100     |
| 2  | 010     | 010     | 200     |
| 3  | 110     | 110     | 010     |
| 4  | 001     | 020     | 110     |
| 5  | 101     | 120     | 210     |
| 6  | 011     | 001     | 001     |
| 7  | 111     | 101     | 101     |
| 8  | 002     | 011     | 201     |
| 9  | 102     | 111     | 011     |
| 10 | 012     | 021     | 111     |
| 11 | 112     | 121     | 211     |

Table 1: **Base permutations.** The table shows the coefficients of integers in  $[0, 12)$  (in rows) factored according to different prime factor basis permutations (in columns).

|    | $s_1 = (2, 6)$         | $s_2 = (3, 4)$         |
|----|------------------------|------------------------|
| 0  | $\langle 0, 0 \rangle$ | $\langle 0, 0 \rangle$ |
| 1  | $\langle 1, 1 \rangle$ | $\langle 1, 1 \rangle$ |
| 2  | $\langle 0, 2 \rangle$ | $\langle 2, 2 \rangle$ |
| 3  | $\langle 1, 3 \rangle$ | $\langle 0, 3 \rangle$ |
| 4  | $\langle 0, 4 \rangle$ | $\langle 1, 0 \rangle$ |
| 5  | $\langle 1, 5 \rangle$ | $\langle 2, 1 \rangle$ |
| 6  | $\langle 0, 0 \rangle$ | $\langle 0, 2 \rangle$ |
| 7  | $\langle 1, 1 \rangle$ | $\langle 1, 3 \rangle$ |
| 8  | $\langle 0, 2 \rangle$ | $\langle 2, 0 \rangle$ |
| 9  | $\langle 1, 3 \rangle$ | $\langle 0, 1 \rangle$ |
| 10 | $\langle 0, 4 \rangle$ | $\langle 1, 2 \rangle$ |
| 11 | $\langle 1, 5 \rangle$ | $\langle 2, 3 \rangle$ |

Table 2: **Choice of moduli.** Remainder tuples  $\langle i \bmod n_1, \dots, i \bmod n_m \rangle$  for  $i \in \mathbb{N}_{<12}$  given different moduli  $s_1 = (2, 6)$  and  $s_2 = (3, 4)$ . Using  $s_1$ , tuples start repeating at the sixth position. Using  $s_2$ , tuples remain unique up to index 11.

### 3.3 Chinese Remainder Theorem

Let  $n_1, \dots, n_m$  be integers, called moduli, all greater than one. Consider the sequence of remainder tuples generated by

$$T = (\langle i \bmod n_1, \dots, i \bmod n_m \rangle)_{i \in \mathbb{N}, 0 \leq i < K},$$

where  $K$  denotes the largest integer such that  $T$  does not contain duplicate entries. The mapping from  $i \mapsto \langle i \bmod n_1, \dots, i \bmod n_m \rangle$  forms a bijection over the domain  $i \in [0, K)$ . When  $(n_1, \dots, n_m)$  are relatively prime, i.e

$$\gcd(n_i, n_j) = 1 \quad 1 \leq i < j \leq m,$$

then  $K$  is maximized and found to be  $K = \prod_{i=1}^m n_i$ . Table 2 illustrates how the choice of moduli affects the uniqueness of remainder tuples.

The Chinese Remainder Theorem (CRT) is a statement about inverting the aforementioned bijection: it asserts that if  $n_1, \dots, n_m$  are relatively prime, an unknown integer  $x$  can be determined from its respective remainders  $t_i = x \bmod n_i$ . These remainder statements form a system of  $m$  parallel congruences

$$x \equiv t_1 \pmod{n_1} \tag{2}$$

$$x \equiv t_2 \pmod{n_2} \tag{3}$$

$$\dots \tag{4}$$

$$x \equiv t_m \pmod{n_m}, \tag{5}$$

that determine  $x$  up to multiples of  $K = \prod_{i=1}^m n_i$ . For  $x < K$  the solution is unique.

We solve for  $x$  by constructing it from a sum of  $m$  terms. The terms are designed so that when applying  $(\text{mod } n_i)$  all terms  $j \neq i$  will vanish but  $i$ -th term will evaluate to  $t_i$ . Let  $K = \prod_{i=1}^m n_i$  as before and recall that  $\text{gcd}(a, b)$  can be written as

$$\text{gcd}(a, b) = ra + sb,$$

for integers  $r, s$  (which can always be found using the Extended Euclidean algorithm). When  $a, b$  are relatively prime this simplifies to

$$1 = ra + sb.$$

Substituting  $a = n_i$  and  $b = K/n_i$  (also co-prime) to get  $1 = rn_i + sK/n_i$ . Applying  $(\text{mod } n_i)$  and distributing we get

$$1 \text{ mod } n_i = rn_i \text{ mod } n_i + sK/n_i \text{ mod } n_i.$$

Since  $rn_i \text{ mod } n_i = 0$  we end up with  $1 = sK/n_i \text{ mod } n_i$ . Define  $e_i = sK/n_i$  which can take on one of two values:

$$e_i = \begin{cases} 1 & (\text{mod } n_i) \\ 0 & (\text{mod } n_j), j \neq i \end{cases}.$$

The unknown  $x$  can now be written as the following sum

$$x = e_1 t_1 + \dots + e_m t_m.$$

Indeed,  $x$  has the desired property of  $x \equiv t_i \pmod{n_i}$ :

$$\begin{aligned} x \text{ mod } n_i &= [e_1 t_1 + \dots + e_m t_m] \text{ mod } n_i \\ &= [0 + \dots + (e_i t_i) \text{ mod } n_i + 0 \dots + 0] \text{ mod } n_i \\ &= [(e_i \text{ mod } n_i)(t_i \text{ mod } n_i)] \text{ mod } n_i \\ &= [1 * (t_i \text{ mod } n_i)] \text{ mod } n_i \\ &= t_i \text{ mod } n_i. \end{aligned}$$

## 4 Codec Principles

The Anoto<sup>®</sup> coding uses four different symbols to encode 2 bits of information (one bit per canonical 2D direction) per dot. The preferred way to represent these symbols is by offsetting dots from their nominal raster position in one of four cardinal directions as depicted in Figure 1.

The coding identifies every 2D location based on of two pieces of information: (i) its section given by *section coordinates*  $\langle u, v \rangle$  and (ii) its position given by *position coordinates*  $\langle x, y \rangle$  within that section. In the preferred embodiment of the encoding, this results in  $63^2$  possible sections, each comprising more than

$1.7 \times 10^{17}$  2D positions. In total,  $6.7 \times 10^{20}$  unique 2D locations can be encoded. Position and section coordinates are uniquely recoverable from any  $6 \times 6$  array of dots, provided the array does not span the boundary of a section. We first start our presentation on decoding and encoding principles by using the preferred embodiment [7–10] and later generalize it to create custom codes.

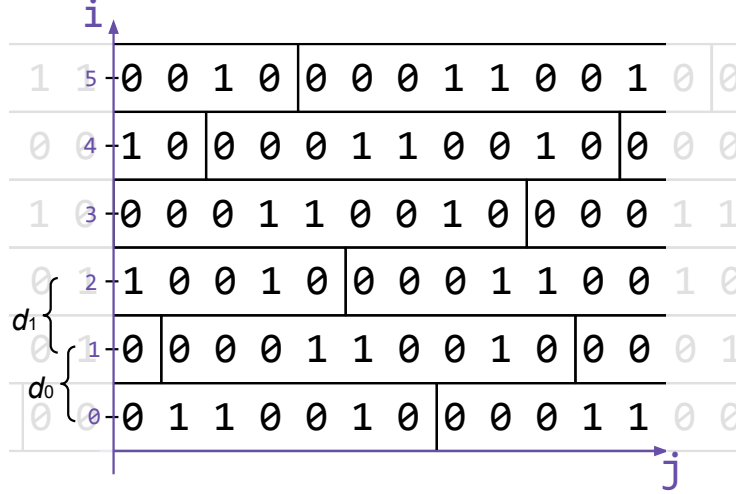


Fig. 2: **Coding principles.** Encoding along one direction (here  $i$ ) involves repeating a periodic quasi de Bruijn sequence (MNS) along the orthogonal direction (here  $j$ ). Consecutive columns differ by varying the sequence start offset. The modular difference between two offsets of adjacent columns is denoted  $d_i$ . For illustration purposes, the length of the MNS is made shorter than the actual sequence used in the Anoto<sup>®</sup> code.

#### 4.1 Decoding

To determine the 2D location associated with a given  $6 \times 6$  sub-array one proceeds as follows: First, each of the  $6 \times 6$  symbols is mapped to two bits of information, one for the  $x$ - and one for the  $y$ -direction. Next, the position coordinates  $\langle x, y \rangle$  are determined by independently processing the resulting bits per dimension. Given the position coordinates  $\langle x, y \rangle$ , the section coordinates  $\langle u, v \rangle$  can be uniquely recovered. When the pattern was rotated by more than  $45^\circ$  during sensor readout, one of the above steps will fail to generate valid results. In this case the canonical pattern orientation must be determined prior to converting the symbols into bits.

**Position Coordinates** This section describes the procedure to calculate the position coordinates  $\langle x, y \rangle$  given a symbol matrix  $\mathbf{S} \in \{N, E, S, W\}^{6 \times 6}$ , that

holds a directional offset per dot. The position coordinates  $\langle x, y \rangle$  are with respect to the top-left element of  $\mathbf{S}$  as shown in Figure 1. Since the position decoding is separable and follows the same scheme for each of the two directions, we choose the  $x$  direction to discuss the decoding in detail.

First, each symbol of  $\mathbf{S}$  is mapped to two separate bits using a lookup table 3. This generates the bit matrix  $\mathbf{P} \in \{0, 1\}^{6 \times 6 \times 2}$ . Define  $\mathbf{P}_x := \mathbf{P}_{:, :, 0}$  to be the matrix of bits related to  $x$ -direction of  $\mathbf{P}$  and let  $\mathbf{x}_i$  be the  $i$ -th column of  $\mathbf{P}_x$ . Each of the six columns  $(\mathbf{x}_1, \dots, \mathbf{x}_6)$  contains a binary string of length 6 which is a sub-string of a quasi de Bruijn sequence (see Section 3.1). This sequence, called the Main Number Sequence (MNS)  $\in QB(2, 6, 63)$ , is of order 6 and length 63 (refer to Appendix A for its values). Each position  $x$  repeats the same MNS along the  $y$ -direction but varies its start offset  $o_x^u$  at  $y = 0$  based on the section coordinate  $u$ . This is depicted in Figure 2. Therefore, the six columns  $(\mathbf{x}_1, \dots, \mathbf{x}_6)$  map to six, in general different, partial sequence indices  $(p_1, \dots, p_6)$ . These indices give raise to five difference values  $(d_1, \dots, d_5)$ , with  $d_i = (p_i - p_{i+1}) \bmod 63$ . For error correction purposes, out of the 63 possible difference values only 54 values  $d_i \in [5, 58]$  are used during encoding.

In principle, each tuple of five consecutive difference values  $(d_1, \dots, d_5)$  already uniquely identifies the position  $x$  if we let  $x$  be the index of the partial sequence  $(d_1, \dots, d_5)$  in a de Bruijn sequence  $\in B(54, 5)$ . This allows for  $54^5 = 459\,165\,024$  different  $x$  positions to be coded, however requires at least 450 MB of memory to store the sequence. To keep the memory footprint manageable, the Anoto<sup>®</sup> coding applies the following additional decomposition: the 54 possible values of  $d_i \in [5, 59]$  are projected onto four prime factor bases given by  $54 = 3 \times 3 \times 2 \times 3$ . Using Equation 1 we represent each  $d_i$  by four coefficients  $(c_1, c_2, c_3, c_4)$  using the polynomial

$$d_i = 5 + c_1 + 3c_2 + 3^2c_3 + (3^2 \cdot 2)c_4, \quad (6)$$

with  $c_1 \in [0, 3), c_2 \in [0, 3), c_3 \in [0, 2)$  and  $c_4 \in [0, 3)$ . These coefficients can be arranged in a coefficient matrix  $\mathbf{C} \in \mathbb{N}^{5 \times 4}$  with column vectors  $\mathbf{c}_i$  holding the  $i$ -th basis coefficients for five consecutive difference values. Each  $\mathbf{c}_i$  is associated with a unique index  $r_i$ , corresponding to the place of the sub-sequence in one of four additional quasi de Bruijn sequences. These sequences are termed Secondary Number Sequences (SNSs) and selected from  $A_1 \in QB(3, 5, 236)$ ,  $A_2 \in QB(3, 5, 233)$ ,  $A_3 \in QB(2, 5, 31)$  and  $A_4 \in QB(3, 5, 241)$  (see Appendix A for actual values). The lengths of the SNS are chosen to be relatively prime, so that the position  $x \in [0, 236 \cdot 233 \cdot 31 \cdot 241)$  can be uniquely computed from  $(r_1, \dots, r_4)$  using the CRT discussed in Section 3.3.

The  $y$  position coordinate can then be determined in a similar fashion by considering the rows of  $\mathbf{P}_y := \mathbf{P}_{:, :, 1}$ . The reason for being able to compute the  $x$  position independently of the  $y$  position is the following: when shifting  $\mathbf{S}$  along the  $y$ -direction one will in general find changing MNS indices  $(p'_1, \dots, p'_6)$ , but the computed difference values  $(d_1, \dots, d_5)$  will remain constant due to modular arithmetic. A similar argument holds for shifting  $\mathbf{S}$  along the  $x$ -direction when decoding for the  $y$  position. Hence decoding the  $x$  position coordinate is independent of the actual  $y$  position and vice versa.

Assuming one byte of memory per sequence character, the described coding requires less than 1 kB of decoding memory compared to 450 MB the naive approach.

| Symbol  | $x$ | $y$ |
|---------|-----|-----|
| N(orth) | 0   | 0   |
| E(ast)  | 0   | 1   |
| S(outh) | 1   | 1   |
| W(est)  | 1   | 0   |

Table 3: **Displacement mapping.** Each possible displacement from a canonical dot position corresponds to one directional symbol and is also associated with a pair of unique  $x$  and  $y$  bits.

**Section Coordinates** After the position coordinates  $\langle x, y \rangle$  have been computed from  $\mathbf{P}$ , one can solve for the unknown section coordinates  $\langle u, v \rangle$ . The section coordinates determine the MNS start offsets  $o_x^u, o_y^v$  at  $y = 0, x = 0$ . In the preferred Anoto<sup>®</sup> embodiment, there are in total 63 possible start offsets per direction. In the following, we will pick again the  $x$ -direction to detail our arguments.

Let  $p_1$  be the MNS index corresponding to the first column  $\mathbf{x}_1$  of  $\mathbf{P}_x$ . Using the decoded position  $y$ , the start offset of MNS at column  $x$  in the current section  $u$  is found to be  $o_x^u = (p_1 - y) \bmod 63$ . Suppose we are given  $o_x^0$ , the start offset of the MNS at the same position  $x$  but in the first section. Then, the modular difference between  $o_x^u$  and  $o_x^0$  gives the section coordinate  $u = (o_x^u - o_x^0) \bmod 63$ . By substitution this simplifies to

$$u = (p_1 - y - o_x^0) \bmod 63.$$

Finally, the value  $o_x^0$  can be solved for via integration using Equation 8 and  $o_0^u = 0$ . The section coordinate  $v$  can be found in a similar fashion.

**Pattern Rotations** When the pattern is rotated by more than 45° during sensor readout, decoding  $\mathbf{S} \in \{N, E, S, W\}^{6 \times 6}$  produces incorrect results. Hence, the canonical orientation of the pattern needs to be determined first. This is done in steps of 90° which leads to four possible rotations  $\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$ . In Anoto<sup>®</sup>, the MNS is constructed such that for slightly larger readout matrices (e.g.  $8 \times 8$ ), the probability of successfully finding partial sequences encoded in the columns and rows of  $\mathbf{P}_x$  and  $\mathbf{P}_y$ , respectively, decreases if the pattern orientation is wrong. To determine the canonical orientation, a straightforward selection mechanism can be used that tests out all four possible orientations.

Once the correct pattern orientation is found and corrected for, decoding can be performed using a  $\mathbf{P} \in \mathbb{N}^{6 \times 6 \times 2}$  matrix. A detail worth mentioning concerns the twofold effect of applying a counter-clockwise (ccw)  $90^\circ$  rotation to  $\mathbf{S}$ : (i) the elements of the matrix rotate counter-clockwise (ii) the displacement symbol at each location (except at a possible center element) changes in ccw-order. These effects accumulate in having the bits of  $\mathbf{P}$  reversed and flipped along at least one of its axis.

## 4.2 Encoding Process

This section describes the steps involved in generating a pattern matrix  $\mathbf{G} \in \{N, E, S, W\}^{H \times W}$  from shape  $\langle H, W \rangle$  and section coordinates  $\langle u, v \rangle$ .

With a bit of abuse of notation, let us first discuss the generation of the bit matrix  $\mathbf{P}_x \in \{0, 1\}^{H \times W}$ . To populate  $\mathbf{P}_x$  we need to find the MNS start offsets  $o_x^u$  for  $x \in [0, W)$  and repeat the MNS accordingly along the columns of  $\mathbf{P}_x$  until  $y = H - 1$  is reached. The offset  $o_x^u$  can be found recursively via the following integration rule

$$o_x^u = (o_{x-1}^u + d_{x-1}) \bmod 63 \quad (7)$$

$$o_0^u = u. \quad (8)$$

The difference value  $d_{x-1}$  is found by first determining the remainders  $(r_1, \dots, r_4)$  of the integer division of  $x - 1$  by the lengths of each SNS. Next, the remainders are used to index four sub-strings of length five from each of the four SNS. These sub-strings constitute the columns of the coefficient matrix  $\mathbf{C} \in \mathbb{N}^{5 \times 4}$  introduced earlier. By construction, the first row of  $\mathbf{C}$  holds the prime factor basis coefficients for  $d_{x-1}$ . Applying Equation 6 allows us to reconstruct the number  $d_{x-1} \in [5, 58]$ . The bit matrix  $\mathbf{P}_y$  can be computed in a similar fashion.

Finally, we construct  $\mathbf{G}$  by converting each pair of bits in  $\mathbf{P}_x$  and  $\mathbf{P}_y$  to one of the displacement symbols  $\in \{N, E, S, W\}$  using lookup table 3.

## 4.3 Generalization

In the previous sections the preferred Anoto<sup>®</sup> embodiment was used to explain the inner workings of the coding in a more tangible and understandable way. The underlying coding and decoding principles, however, are also transferable to other embodiments. This is useful, for example, to build codings that only a  $4 \times 4$  matrix to decode a unique position. To generalize the coding the following code components need to be selected appropriately:

**MNS** The MNS is a binary cut-down de Bruijn sequence  $B(2, n, m)$  of order  $n$  and length  $m$ . The length  $m$  sets the range for possible sub-string positions  $p_i \in [0, m)$ . The chosen order  $n$  defines the minimum number of rows of  $\mathbf{S}$  during decoding. For symmetry reasons, we assume  $\mathbf{S}$  to be square and hence there will be  $n - 1$  possible difference values  $d_i \in [0, m)$  for  $n$  columns in  $\mathbf{S}$ . The maximum number of unique positions per direction and section is therefore  $m^{n-1}$ .

**Range and Prime Factors** The range of possible difference values  $[0, m)$  is limited to a chosen interval  $[l, u)$  for two reasons: (i) to lower memory requirements and (ii) for error detection purposes upon decoding. The latter is obtained by not allowing difference values that would lead to symbol matrices with symmetrical patterns. The former is realized by tuning the interval length  $u - l$  to a number that can be decomposed into  $k$  prime factors  $(f_1, \dots, f_k)$ . By representing each difference value by its coefficients with respect to the resulting prime factor bases, the memory requirement for storing the final SNS de Bruijn sequences is negligible. The  $k$  prime factors  $(f_1, \dots, f_k)$  can be used to define a basis for numbers in  $[0, u - l)$  as described in Section 3.2. Each difference value  $d_i$  can hence be represented as  $d_i = \sum_{i=1}^k b_i c_i + l$  with coefficients  $c_i$ . From  $n - 1$  difference values we therefore compute  $(n - 1) \cdot k$  coefficients that we arrange in a matrix  $\mathbf{C} \in \mathbb{N}^{(n-1) \times k}$ .

**SNS** For a successful decoding we require  $k$  cut-down de Bruijn sequences of order  $n - 1$ . The alphabet size for the  $i$ -th sequence is  $[0, f_i)$ . We select these sequences such that their lengths  $(l_1, \dots, l_k)$  are relatively prime, which ensures that each position  $x \in [0, l_1 \cdot l_2 \cdots l_k)$  uniquely maps to a tuple of secondary number indices  $\langle r_1, \dots, r_k \rangle$  from which  $x$  can later be recovered using the algorithm presented in Section 3.3.

## 5 py-microdots

This section introduces **py-microdots**<sup>2</sup>, a Python library to work with Anoto<sup>®</sup> patterns. The scope of **py-microdots** is centered on the pattern encoding and decoding parts, as shown in Figure 3. Image processing and plotting functionality can vary greatly depending on the target application and have therefore not been included in the feature set.

The latest version of **py-microdots** can be easily obtained via

---

```
pip install https://github.com/cheind/py-microdots.git
```

---

To use the library one first defines a specific codec configuration to be used. To simplify the process, the library exposes a set of pre-configured codecs. For example, to work with the preferred Anoto<sup>®</sup> embodiment with a correction made to the last secondary sequence (see Appendix A for details) use

---

```
# Import the library
import microdots as mdots

# Use the default embodiment with A4 sequence fixed
codec = mdots.anoto_6x6_a4_fixed
```

---

The following code shows how to encode a pattern matrix  $\mathbf{G}$  with shape  $\langle 9, 16 \rangle$  and section coordinates  $\langle 10, 2 \rangle$

---

<sup>2</sup> <https://github.com/cheind/py-microdots>

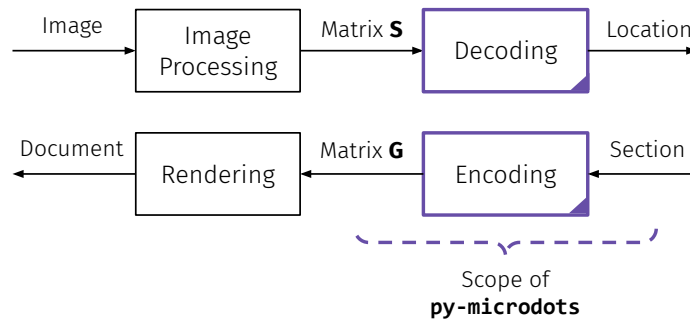


Fig. 3: **Scope of py-microdots.** The library `py-microdots` focuses on encoding locations into and decoding location information from bit matrices. During decoding, a partial symbol matrix **S** is converted into location information (position, section, and rotation parts). Encoding generates a pattern matrix **G** of the desired shape which may then be superimposed onto a document in a further step.

---

```

# Generate a bit-matrix for section (10,2)
G = codec.encode_bitmatrix(shape=(9, 16), section=(10, 2))

```

---

For debugging purposes, one may plot **G** in appearance similar to what is shown in Figure 1 using the following code

---

```

import matplotlib.pyplot as plt

# Render dots
fig, ax = plt.subplots()
mdots.draw_dots(G, grid_size=1.0, show_grid=True, ax=ax)
fig.savefig("dots.pdf")
plt.close(fig)

```

---

To decode the location given a partial observed matrix **P** three functionalities in accordance with information given in Section 4.1 are provided

---

```

# Decode a partial matrix
S = G[3:3+6, 7:7+6]

rot = codec.decode_rotation(S)
pos = codec.decode_position(S)
sec = codec.decode_section(S, pos=pos)
print("pos:", pos, "sec:", sec, "rot:", rot)
# > pos: (7, 3) sec: (10, 2) rot: 0

```

---

Here, `pos` contains the *position coordinates*  $\langle x, y \rangle$ , `sec` holds the *section coordinates*  $\langle u, v \rangle$  and `rot` contains the pattern orientation in steps of  $90^\circ$  following a counter-clockwise rotation.

To instance a custom codec, one creates an instance of `AnotoCodec` and provides the required codec information (see in Section 4.3 for details). For example, a codec version that requires a  $4 \times 4$  partial window to be observed during decoding may be instanced as follows

---

```
# Import sequences
from microdots.mini_sequences import MNS, A1, A2

# Instantiate the codec
codec4x4 = mdots.AnotoCodec(
    mns=MNS,
    mns_order=4,
    sns=[A1, A2],
    pfactors=(3, 5),
    delta_range=(1, 15),
)
```

---

This configuration uses a MNS of length  $2^4$  and limits possible difference values to  $[1, 15]$ . The number 15 decomposes into prime factors  $(3, 5)$  and we select  $A_1 \in QB(3, 3, 27)$  and  $A_2 \in QB(5, 3, 125)$  as SNS. These sequences are predefined by the library and set to be

---

```
import numpy as np

# Length 16 (2**4)
MNS = np.array([
    0,0,0,0,1,0,0,1,1,0,1,0,1,1,1,1,
], dtype=np.int8)

# Length 27
A1 = np.array([
    0,0,0,1,0,0,2,0,1,1,0,1,2,0,2,1,0,2,2,1,1,1,2,1,2,2,2
], dtype=np.int8)

# Length 125
A2 = np.array([
    0,0,0,1,0,0,2,0,0,3,0,0,4,0,1,1,0,1,2,0,1,3,0,1,4,0,2,
    1,0,2,2,0,2,3,0,2,4,0,3,1,0,3,2,0,3,3,0,3,4,0,4,1,0,4,
    2,0,4,3,0,4,4,1,1,1,2,1,1,3,1,1,4,1,2,2,1,2,3,1,2,4,1,
    3,2,1,3,3,1,3,4,1,4,2,1,4,3,1,4,4,2,2,2,3,2,2,4,2,3,3,
    2,3,4,2,4,3,2,4,4,3,3,3,4,3,4,4,4
], dtype=np.int8)
```

---

These quasi de Bruijn sequences were generated using the method proposed by [Cameron et al. \[2\]](#). The resulting codec configuration provides  $3375 \times 3375$

unique position coordinates and  $16 \times 16$  section possibilities and can be decoded by observing a  $4 \times 4$  grid of dots.

## 6 Results

### 6.1 Preferred Embodiment

During our research we noticed that  $A_4$ , the last of four SNSs used in the default embodiment of the Anoto<sup>®</sup> code, is supposed to be from  $QB(3, 5, 241)$ , but the original patents [7] list only 240 digits of the sequence  $A_4$ . This has prompted several authors of previous works [4, 12] to append the digit 1 to the sequence to complete it. However, our investigations reveal that this particular sequence does not satisfy the de Bruijn property of unique sub-strings. It contains 6 sub-string collisions that are either caused by the modification or were already present before. These collisions are listed in Table 4. Using this sequences ultimately leads to decoding errors, the first is encountered at at position  $x = 217$ . To mitigate the issue, we provide provide an alternative quasi de Bruijn sequence with desired properties named  $A_{4b}$ . Both sequences,  $A_4$  and  $A_{4b}$  can be found in Appendix A.

| Sub-string      | Earliest Index | Colliding Index |
|-----------------|----------------|-----------------|
| (1, 1, 1, 0, 0) | 82             | 238             |
| (1, 1, 2, 1, 2) | 195            | 217             |
| (1, 2, 1, 2, 2) | 198            | 218             |
| (2, 1, 2, 2, 2) | 199            | 219             |
| (2, 2, 1, 1, 1) | 170            | 236             |
| (2, 1, 1, 1, 0) | 177            | 237             |

Table 4: **String collisions.** Sub-string collision indices found in the Secondary Number Sequence  $A_4$  of the preferred Anoto<sup>®</sup> embodiment, after completing using a 1 as last digit. Indices starting at zero.

### 6.2 Timings

An important aspect of py-microdots with respect to real-world applications are the run-times for encoding and decoding. Table 5 lists timings for various steps in the library. In particular, the following tasks are considered: (i) encoding an A0 841 mm  $\times$  1188 mm paper at a grid resolution of 0.3 mm, (ii) decoding position coordinates given a  $6 \times 6$  matrix  $\mathbf{P}$ , (iii) decoding section coordinates given a  $6 \times 6$  matrix  $\mathbf{P}$ , and (iv) decoding pattern orientation given a  $8 \times 8$  matrix  $\mathbf{P}$ . Decoding section coordinates requires an integration step over all positions  $[0, x]$ , and is hence typically an order of magnitude slower than position decoding.

| Task                | Timing       |
|---------------------|--------------|
| Encoding A0 paper   | 6406 $\mu$ s |
| Decode 6x6 position | 41 $\mu$ s   |
| Decode 6x6 section  | 702 $\mu$ s  |
| Decode 8x8 rotation | 12 $\mu$ s   |

Table 5: **Performance timings.** Timings are the average over 100 runs. All benchmarks were performed on a i9-12900K CPU with 64 GB of RAM.

## 7 Conclusion

This work gave an in-depth review of the Anoto<sup>®</sup> structured dot pattern - a technique for position encoding that requires only a small portion of pattern to be observed in order to uniquely determine the position of a device over large areas. Starting from the preferred embodiment of the coding, we generalized the underlying principles to generate application tailored variants of the coding. In addition, we complement our findings with the first pure Python library called `py-microdots` to make encoding/decoding easily accessible to practitioners.

While we have dealt extensively with the theoretical aspects of coding, many practical details are left to future work. An interesting research direction is to aim at generalizing image feature extraction through a data-driven approach. We also intend to apply the coding to the task of camera calibration, camera-robot calibration, and indoor localization.

## Acknowledgments

We would like to thank Sebastian Zambal for helpful suggestions and Sezgin Kutlu for reviewing the paper draft. The project is generously supported in part by the European Union’s Horizon 2020 research and innovation programme within the project CONCERT (grant number 101016007).

## Bibliography

- [1] Aboufadel, E., Armstrong, T., Smietana, E.: Position coding. arXiv preprint arXiv:0706.0869 (2007)
- [2] Cameron, B., Gündoğan, A., Sawada, J.: Cut-down de bruijn sequences. arXiv preprint arXiv:2205.02815 (2022)
- [3] Flaherty, G.: Basic Anoto Outline. <http://freedot.sourceforge.net/> (2005)
- [4] Hostettler, L., Özgür, A., Lemaignan, S., Dillenbourg, P., Mondada, F.: Real-time high-accuracy 2d localization with structured patterns. In: 2016 IEEE International Conference on Robotics and Automation (ICRA), pp. 4536–4543, IEEE (2016)
- [5] Ireland, K., Rosen, M.I., Rosen, M.: A classical introduction to modern number theory, vol. 84. Springer Science & Business Media (1990)
- [6] Manin, Y.I., Panchishkin, A.A.: Introduction to modern number theory, vol. 49. Springer (2005)
- [7] Pettersson, M.P.: Method and device for decoding a position-coding pattern. EP1451767A1 (2004), URL <https://patents.google.com/patent/EP1451767A1>
- [8] Pettersson, M.P., Edsö, T.: Determination of a position code. EP1224606 (2002), URL <https://patents.google.com/patent/EP1224606B1>
- [9] Pettersson, M.P., Edsö, T.: Encoded paper for optical reading. EP1222604 (2002), URL <https://patents.google.com/patent/EP1222604B1>
- [10] Pettersson, M.P., Edsö, T.: Orientation discrimination in a position-coding pattern. US7248250B2 (2007), URL <https://patents.google.com/patent/US7248250/en>
- [11] Zhang, C., Chen, L., Huang, W., Guo, Z., Wang, Q.: Apple stem-end/calyx identification using a speckle-array encoding pattern. In: 2012 IEEE 11th International Conference on Signal Processing, vol. 2, pp. 1110–1114, IEEE (2012)
- [12] Özgür, A.: Cellulo: Tangible haptic swarm robots for learning (2018), doi: 10.5075/epfl-thesis-8241, URL <http://infoscience.epfl.ch/record/234374>

## A Anoto Sequences

This section lists the De Bruijn sequences used in the preferred embodiment of the Anoto<sup>®</sup> code, describes the problems related to them and points out alternatives.

### A.1 Main Number Sequence (MNS)

The MNS is a quasi de Bruijn sequence of order 6 and length 63 over the binary alphabet  $QB(2, 6, 63)$  chosen to be

MNS = (0,0,0,0,0,0,1,0,0,1,1,1,1,1,0,1,0,0,  
1,0,0,0,0,1,1,1,0,1,1,1,0,0,1,0,1,0,  
1,0,0,0,1,0,1,1,0,1,1,0,0,1,1,0,1,0,  
1,1,1,1,0,0,0,1,1).

### A.2 SNSs

In total there are four secondary number sequences ( $A_1, A_2, A_3, A_4$ ) corresponding to four prime factor bases (3, 3, 2, 3) that generate integers in the range  $[0, 54)$ . Each sequence is a quasi de Bruijn sequence of order 5. Their lengths are chosen to be relatively co-prime to ensure a maximum number of unique remainder tuples being generated and allow the CRT to work.

$A_1$  follows a  $QB(3, 5, 236)$  and is chosen to be

$A_1$  = (0,0,0,0,0,1,0,0,0,0,2,0,1,0,0,1,0,1,0,  
0,2,0,0,0,1,1,0,0,0,1,2,0,0,1,0,2,0,0,  
2,0,2,0,1,1,0,1,0,1,1,0,2,0,1,2,0,1,0,  
1,2,0,2,1,0,0,1,1,1,0,1,1,1,1,0,2,1,0,  
1,0,2,1,1,0,0,1,2,1,0,1,1,2,0,0,0,2,1,  
0,2,0,2,1,1,1,0,0,2,1,2,0,1,1,1,2,0,2,  
0,0,1,1,2,1,0,0,0,2,2,0,1,0,2,2,0,0,1,  
2,2,0,2,0,2,2,1,0,1,2,1,2,1,0,2,1,2,1,  
1,0,2,2,1,2,1,2,0,2,2,0,2,2,2,0,1,1,2,  
2,1,1,0,1,2,2,2,2,1,2,0,0,2,2,1,1,2,1,  
2,2,1,0,2,2,2,2,2,0,2,1,2,2,2,1,1,1,2,  
1,1,2,0,1,2,2,1,2,2,0,1,2,1,1,1,1,2,2,  
2,0,0,2,1,1,2,2).

$A_2$  follows a  $QB(3, 5, 233)$  and is chosen to be

$A_2$  = (0,0,0,0,0,1,0,0,0,0,2,0,1,0,0,1,0,1,0,  
1,1,0,0,0,1,1,1,1,0,0,1,1,0,1,0,0,2,0,  
0,0,1,2,0,1,0,1,2,1,0,0,0,2,1,1,1,0,1,  
1,1,0,2,1,0,0,1,2,1,2,1,0,1,0,2,0,1,1,  
0,2,0,0,1,0,2,1,2,0,0,0,2,2,0,0,1,1,2,  
0,2,0,0,2,0,2,0,1,2,0,0,2,2,1,1,0,0,2,

1,0,1,1,2,1,0,2,0,2,2,1,0,0,2,2,2,1,0,  
 1,2,2,0,0,2,1,2,2,1,1,1,1,1,2,0,0,1,2,  
 2,1,2,0,1,1,1,2,1,1,2,0,1,2,1,1,1,2,2,  
 0,2,2,0,1,1,2,2,2,2,1,2,1,2,2,0,1,2,2,  
 2,0,2,0,2,1,1,2,2,1,0,2,2,0,2,1,0,2,1,  
 1,0,2,2,2,2,0,1,0,2,2,1,2,2,2,1,1,2,1,  
 2,0,2,2,2).

$A_3$  follows a  $QB(2, 5, 31)$  and is chosen to be

$A_3 = (0,0,0,0,0,1,0,0,1,1,0,0,0,1,1,1,1,0,0,$   
 $1,0,1,0,1,1,0,1,1,1,0,1).$

Finally  $A_4$  is supposed to be from  $QB(3, 5, 241)$ , but the original patents [7] list only 240 digits of the sequence  $A_4$ . This has prompted several works [4, 12] to append a 1 to the sequence to complete it. The resulting sequence is shown below

$A_4 = (0,0,0,0,0,1,0,2,0,0,0,0,2,0,0,2,0,1,0,$   
 $0,0,1,1,2,0,0,0,1,2,0,0,2,1,0,0,0,2,1,$   
 $1,2,0,1,0,1,0,0,1,2,1,0,0,1,0,0,2,2,0,$   
 $0,0,2,2,1,0,2,0,1,1,0,0,1,1,1,0,1,0,1,$   
 $1,0,1,2,0,1,1,1,1,0,0,2,0,2,0,1,2,0,2,$   
 $2,0,1,0,2,1,0,1,2,1,1,0,1,1,1,2,2,0,0,$   
 $1,0,1,2,2,2,0,0,2,2,2,0,1,2,1,2,0,2,0,$   
 $0,1,2,2,0,1,1,2,1,0,2,1,1,0,2,0,2,1,2,$   
 $0,0,1,1,0,2,1,2,1,0,1,0,2,2,0,2,1,0,2,$   
 $2,1,1,1,2,0,2,1,1,1,0,2,2,2,2,0,2,0,2,$   
 $2,1,2,1,1,1,1,2,1,2,1,2,2,2,1,0,0,2,1,$   
 $2,2,1,0,1,1,2,2,1,1,2,1,2,2,2,2,1,2,0,$   
 $1,2,2,1,2,2,0,2,2,2,1,1,1).$

As mentioned in Section 6 this sequence does not represent a valid quasi de Bruijn sequence because of multiple sub-string collisions. Instead, we propose using an alternative de Bruijn sequence  $A_{4b}$  with the following values

$A_{4b} = (0,0,0,0,2,2,2,2,0,2,2,2,1,0,2,2,2,0,0,$   
 $2,2,1,2,0,2,2,1,1,0,2,2,1,0,0,2,2,0,0,$   
 $0,2,1,2,2,0,2,1,2,1,0,2,1,2,0,0,2,1,1,$   
 $2,0,2,1,1,1,0,2,1,1,0,0,2,1,0,0,0,2,0,$   
 $2,2,0,2,0,2,1,0,2,0,2,0,0,2,0,1,0,0,2,$   
 $0,0,0,0,1,2,2,2,0,1,2,2,1,0,1,2,2,0,0,$   
 $1,2,1,2,0,1,2,1,1,0,1,2,1,0,0,1,2,0,0,$   
 $0,1,1,2,2,0,1,1,2,1,0,1,1,2,0,0,1,1,1,$   
 $2,0,1,1,1,1,2,2,2,2,1,2,2,2,1,1,2,2,1,$   
 $1,1,2,1,2,2,1,2,1,2,1,1,2,1,1,1,1,0,$   
 $1,1,1,0,0,1,1,0,0,0,1,0,2,2,0,1,0,2,1,$   
 $0,1,0,2,0,0,1,0,1,2,0,2,0,1,2,0,1,0,1,$   
 $1,0,2,0,1,1,0,1,0,1,0,0,1).$